

วันที่ 6 — วันสุดท้าย

Effective Modelling & Capstone Project

Bounds · Redundant Constraints · Debugging · นำไปใช้จริง



Tight Bounds

solver เร็วขึ้น



Redundant

prune เพิ่ม



Debug

แก้ปัญห าคือ



Capstone

โมเดลจริง

กำหนดการวันที่ 6

09:00–09:45



Variable Bounds

tight bounds ช่วย solver อย่างไร — *before/after* ตัวอย่าง

09:45–10:30



Redundant Constraints

ทำไม *constraint* ที่ 'ซ้ำซ้อน' ช่วยได้ — *lower bound tricks*

10:30–11:00



Debugging Workflow

UNSAT → *Slow* → *Wrong Results* — ขั้นตอนแก้ปัญหา

11:00–12:00



Workshop 6A — Debugging

3 กรณีจริง: *UNSAT*, *Slow*, *Wrong* — หาและแก้ไขปัญหา

13:00–15:30



Capstone Project

สร้างโมเดลจากปัญหาจริงขององค์กร — ทำงานเป็นกลุ่ม

15:30–16:00



นำเสนอ Capstone

แต่ละกลุ่มนำเสนอ 10 นาที — ปัญหา โมเดล ผลลัพธ์

Variable Bounds — ช่วย Solver ตัด Space

Solver ทำงานใน search space ที่กำหนดโดย bounds → bounds แน่น = space เล็ก = เร็วกว่า

✗ Bound กว้างเกินไป

```
% ✗ Hardcode ค่าสูง
var 0..999: start;
var 0..999: makespan;

% Solver ค้นหาใน 0..999

% = 1,000 ค่า ต่อตัวแปร
```

✓ Bound แน่น

```
% ✓ คำนวณ bound จากข้อมูล
int: total=sum(j,t)(d[j,t]);
int: lb=max(j)(sum(t)(d[j,t]));
var 0..total: start;
var lb..total: makespan;

% lb = lower bound ที่รู้แน่ๆ
```



Rule: lower bound = เร็วที่สุดที่หฤษฎีบอก | upper bound = ช้าที่สุดที่สมเหตุสมผล

การคำนวณ Bounds ที่ดี

ปัญหา	Lower Bound (lb)	Upper Bound (ub)
Job Shop makespan	<code>max(j)(sum(t)(d[j,t]))</code>	<code>sum(j,t)(d[j,t])</code>
Bin Packing n_bins	<code>ceil(sum(sizes)/capacity)</code>	<code>n_items</code> (worst case)
Rostering nights	<code>ceil(req_night*n_days/n_emp)</code>	<code>n_days</code> (every night)
VRP total distance	<code>sum(min_dist_to_nearest)</code>	<code>sum(all_dist)</code> (loose)
Scheduling start	<code>0</code>	<code>sum(all_durations) - min_dur</code>

Pattern: `var lb..ub: variable;`

```
int: lb = max(j)(sum(t)(d[j,t])); int: ub = sum(j,t)(d[j,t]);
var lb..ub: makespan; % แทน var 0..999: makespan;
```

Redundant Constraints — เพิ่ม Pruning

Redundant = จริงอยู่แล้วจาก constraint อื่น แต่ช่วย solver ตัดสิ่งที่เป็นไปได้ได้เร็วกว่า

```
% ปัญหา Assignment (แต่ละคนได้งานต่างกัน)
```

```
constraint alldifferent(assigned);
solve minimize sum(w)(cost[w, assigned[w]]);
```

```
% — Redundant constraints เพิ่ม pruning —————
```

```
% Lower bound: ต้นทุนรวมต้องอย่างน้อยเท่ากับ sum ของ min ต่อ worker
```

```
constraint sum(w)(min(t)(cost[w,t])) <= sum(w)(cost[w,assigned[w]]);
```

```
% Lower bound อีกแบบ: sum ของ min ต่อ task
```

```
constraint sum(t)(min(w)(cost[w,t])) <= sum(w)(cost[w,assigned[w]]);
```

ทำไมได้ผล:

Redundant constraint บอก solver

min cost ที่เป็นไปได้ก่อนหาคำตอบ — *prune branches* ที่ *cost* จะเกินได้เลย

ไม่ตัด optimal solution

เพราะ constraint จริงอยู่แล้ว — เป็นแค่ hint ให้ solver ฉลาดขึ้น

Debugging Workflow — 3 ประเภทปัญหา



UNSATIFIABLE

1. เพิ่ม constraint ทีละอย่าง — หว่าอันไหนทำให้ขัดแย้ง
2. ลด scope: เริ่มกับข้อมูลน้อยๆ (2-3 items)
3. คำนวณมือ: ผลลัพธ์ที่เป็นไปได้มีจริงไหม?
4. ผ่อนคลาย constraint แล้วดูว่า UNSAT หายไหม



SLOW / TIMEOUT

1. ตรวจสอบ bounds: กว้างเกินไปไหม? จำนวน lb/ub ที่แน่น
2. เพิ่ม redundant constraints เพื่อช่วย prune
3. เพิ่ม symmetry breaking
4. ลอง solver อื่น: Chuffed, OR-Tools, Gurobi



WRONG RESULT

1. ตรวจสอบ output มือ: จำนวน resource usage แล้วเทียบ
2. ลอง solve satisfy ก่อน ดูว่า model ถูก
3. เพิ่ม assert() ตรวจสอบ data ที่รับมา
4. ตรวจสอบ array index: แกว/คอลัมน์สลับกันไหม?



Workshop 6A — Debugging Challenge

เวลา: 60 นาที | ไฟล์: debug_broken_1.mzn, debug_broken_2.mzn, debug_broken_3.mzn



กรณี 1: UNSATISFIABLE (debug_broken_1.mzn)

Maintenance Scheduling — deadline ที่กำหนดเป็นไปไม่ได้

ภารกิจ: หาว่า deadline ต้องเป็นเท่าไรถึงจะ satisfy? คำนวณมือก่อนแล้วยืนยันด้วย MiniZinc



กรณี 2: ช้ามาก (debug_broken_2.mzn)

Job Shop 6x4 — bounds hardcoded เป็น 200 ทั้งที่คำนวณได้

ภารกิจ: คำนวณ total และ lb ที่แน่น แล้วเปรียบเทียบ solve time



กรณี 3: ผลไม่ถูกต้อง (debug_broken_3.mzn)

Production Planning — model รันได้แต่ resource usage เกิน capacity

ภารกิจ: หา bug แล้วแก้ไข ยืนยันด้วยการคำนวณมือ

ช่วงบ่าย

Capstone Project

— สร้างโมเดลจากงานจริง

- เลือกปัญหาจากองค์กรของท่าน หรือจากโจทย์ที่กำหนด

- ออกแบบ model: variables, constraints, objective

- เขียน MiniZinc โดยใช้ capstone_template.mzn

- ทดสอบ ปรับแต่ง และนำเสนอ

ตัวเลือกโจทย์ Capstone Project

การผลิต

- ▶ Multi-machine Job Shop: 5 jobs, 4 เครื่อง, minimize makespan
- ▶ Production Lot Sizing: วางแผนผลิต 8 สินค้า 4 สัปดาห์
- ▶ Assembly Line Balancing: กระจาย task ลง 4 สถานี

Logistics

- ▶ Multi-Depot VRP: 2 คลัง 15 ลูกค้า 5 รถ
- ▶ Warehouse Slotting: จัดตำแหน่งสินค้า 50 SKU
- ▶ Container Loading: จัดตู้ลดค่าขนส่ง

HR / Workforce

- ▶ Employee Rostering: 15 คน 3กะ 4 สัปดาห์ กฎแรงงาน
- ▶ Workforce Planning: จัด skill mix + fairness
- ▶ Exam Scheduling: จัดสอบ ห้อง + เวลา ไม่ชน

ปัญหาจากองค์กร

- ▶ นำปัญหาที่ท่านเจออยู่ทุกวัน
- ▶ วิทยากรช่วยออกแบบ model ให้
- ▶ ผลลัพธ์นำไปใช้ได้จริงได้เลย

ขั้นตอนสร้างโมเดลจากปัญหาจริง

1

นิยามปัญหา (10 นาที)

อะไรคือ Decision? (ใครทำอะไร เมื่อไร กี่หน่วย) • Objective คืออะไร? (minimize cost / maximize profit) • Constraints มีอะไรบ้าง? (จดทุกข้อที่นึกได้)

2

ออกแบบ Variables (15 นาที)

กำหนด enum สำหรับชื่อสิ่งต่างๆ • กำหนด decision variables + bounds แน่น • เริ่มด้วย solve satisfy ก่อนเสมอ

3

เพิ่ม Constraints ทีละอย่าง (30 นาที)

Constraint สำคัญที่สุดก่อน — ทดสอบทุกครั้ง • ถ้า UNSAT: หา constraint ที่ทำให้ขัดแย้ง • เพิ่ม Objective และ minimize/maximize สุดท้าย

4

ทดสอบและปรับแต่ง (15 นาที)

ตรวจ output: คำนวณมือว่าถูกต้องไหม? • ถ้าซ้ำ: ปรับ bounds, เพิ่ม redundant • เตรียม data file จริง

การนำเสนอ Capstone (10 นาที/กลุ่ม)

2 นาที

ปัญหาคืออะไร?

อธิบายปัญหาจริงในองค์กร — ทำไมต้อง *optimize*?

3 นาที

โมเดล MiniZinc

Variables, Constraints หลัก, *Objective* — แสดงโค้ดสั้นๆ

3 นาที

ผลลัพธ์ที่ได้

คำตอบที่ดีที่สุดคืออะไร? ดีกว่าวิธีเดิมแค่ไหน?

2 นาที

ถาม-ตอบ

สิ่งที่เรียนรู้, ความท้าทาย, ขั้้นต่อไป



focus ที่ปัญหาและผลลัพธ์ — ไม่ต้องอธิบาย *syntax* ทุกบรรทัด

สรุปหลักสูตร 6 วัน

1

วันที่ 1: พื้นฐาน

var, constraint, solve satisfy/minimize/maximize, data files

2

วันที่ 2: Arrays & Enums

forall, sum, array indexing, enum types, bool2int

3

วันที่ 3: Global Constraints

alldifferent, cumulative, table, disjunctive, Job Shop

4

วันที่ 4: Scheduling

predicate, Flexible Job Shop, Rostering, regular

5

วันที่ 5: Routing & Packing

circuit/TSP/VRP, bin_packing_capa, Knapsack

6

วันที่ 6: Effective Modelling

tight bounds, redundant constraints, debugging, Capstone

Quick Reference: Global Constraints ทั้งหมด

Constraint	Include	ใช้กับ	ตัวอย่าง
<code>alldifferent(x)</code>	<code>globals.mzn</code>	Assignment, no-dup	<code>alldifferent(assigned)</code>
<code>cumulative(s,d,r,b)</code>	<code>cumulative.mzn</code>	Resource scheduling	<code>cumulative(start,dur,workers,cap)</code>
<code>disjunctive(s,d)</code>	<code>disjunctive.mzn</code>	Machine no-overlap	<code>disjunctive([s[j],t] j in J],[d[j],t] j in J)</code>
<code>table(x,tuples)</code>	<code>table.mzn</code>	Lookup/spec table	<code>table([mat,hard,wt,cost],mat_table)</code>
<code>regular(seq,tr,q0,F)</code>	<code>regular.mzn</code>	Shift patterns	<code>regular([roster[e,d] d in D],tr,S1,STATE)</code>
<code>circuit(next)</code>	<code>circuit.mzn</code>	TSP/VRP routing	<code>circuit(next)</code>
<code>bin_packing_capa(c,b,w)</code>	<code>bin_packing_capa.mzn</code>	Bin packing	<code>bin_packing_capa(cap,bin_of,sizes)</code>

Checklist ก่อน Submit โมเดล

☐ Bounds แน่น

ไม่ใช่ 0..9999 — คำนวณ lb/ub จากข้อมูลจริง

☐ ข้อมูลขนาดเล็กก่อน

ทดสอบกับ $n=2,3$ ก่อน scale ขึ้น

☐ มี assert() ตรวจสอบ data

บอก error ที่ชัดเจนถ้า data ผิด

☒ solve satisfy ก่อน

ทดสอบว่า model ถูกก่อนเพิ่ม objective

☐ ตรวจสอบ output มือ

คำนวณ resource usage เทียบกับ constraint

☐ Comment อธิบาย constraint

คนอื่น (หรือตัวเองในอีก 1 เดือน) อ่านรู้เรื่อง

ขั้นต่อไปหลังอบรม



แหล่งเรียนรู้เพิ่มเติม

- ▶ docs.minizinc.dev — Official tutorial
- ▶ minizinc.org/challenge — ตัวอย่างโมเดลจาก competition
- ▶ MiniZinc Playground — ทดลองออนไลน์
- ▶ Python + minizinc package — เชื่อม automation
- ▶ Gurobi/CPLEX — สำหรับปัญหาใหญ่ระดับ enterprise



นำไปใช้จริง

- ✓ นำ Capstone model ไปทดสอบกับข้อมูลจริง
- ✓ เพิ่ม solver เชิงพาณิชย์ถ้าปัญหาใหญ่ขึ้น
- ✓ ขยาย model เพิ่ม constraint ใหม่ตามความต้องการ
- ✓ แชรร์ model กับทีม — ทำ documentation
- ✓ ติดต่อวิทยากรถ้าติดปัญหา



ขอแสดงความยินดี!

ท่านสามารถใช้ MiniZinc แก้ปัญหา Optimization
ในงานอุตสาหกรรมได้แล้ว

Variables

Constraints

forall/sum

Arrays

Enum

Global Constraints

cumulative

alldifferent

Job Shop

Rostering

regular

VRP

Bin Packing

Knapsack

Debugging

Quick Reference: Effective Modelling

```
% — TIGHT BOUNDS —————

int: total = sum(j,t)(d[j,t]);      % upper bound
int: lb     = max(j)(sum(t)(d[j,t])); % lower bound
var lb..total: makespan;

% — REDUNDANT CONSTRAINTS —————

% Lower bound: objective >= sum(row_min)
constraint sum(w)(min(t)(cost[w,t])) <= sum(w)(cost[w,assigned[w]]);

% — SYMMETRY BREAKING (examples) —————

% Bin Packing: bin_of[i] <= bin_of[i+1]+1
constraint forall(i in 1..n-1)(bin_of[i] <= bin_of[i+1]+1);

% Job Shop: ถ้า 2 jobs ทำ task เดียวกันพร้อมกัน force j<k
% Assignment: worker 1 ใ้ task หมายเลขน้อยที่สุด
constraint assigned[1] = min(TASKS); % อาจเร็วขึ้น
```

Quick Reference: Debugging

% — UNSATISFIABLE —

% 1. เพิ่ม constraint ที่ละเอียด: เริ่ม solve satisfy ก่อน

`solve satisfy;` % ไม่ minimize ก่อน

% 2. ลด n: เริ่มด้วยข้อมูล 2-3 items

% 3. Comment constraint ออก แล้วเพิ่มที่ละเอียด

% — SLOW SOLVER —

% 1. กำหนด bounds แทน hardcoded

`int: ub = sum(j,t)(d[j,t]);` % แทน var 0..999

% 2. เพิ่ม symmetry breaking

% 3. ลอง solver อื่น: `--solver chuffed` หรือ `or-tools`

% — WRONG RESULTS —

% 1. เพิ่ม `assert()` ตรวจสอบ input data

`constraint assert(n > 0, "n must be positive");`

% 2. เพิ่ม output ตรวจสอบ intermediate values

วันที่ 6: Effective Modelling - Quick Ref: Debugging `min(sum(p)(c[p,r]*x[p])), "\n");`

ตัดสินใจเลือก Pattern ให้ถูก

ต้องการ	Pattern	วันที่เรียน
ทุกค่าต้องต่างกัน	<code>alldifferent(x)</code>	วัน 3
ทรัพยากรไม่เกิน cap ทุกเวลา	<code>cumulative(s,d,r,cap)</code>	วัน 3
เครื่องทีละ 1 job	<code>disjunctive(s,d)</code>	วัน 3
เลือกจาก tuple ที่กำหนด	<code>table(x, allowed)</code>	วัน 3
บังคับ shift pattern	<code>regular(seq,tr,q0,F)</code>	วัน 4
Hamiltonian cycle (TSP/VRP)	<code>circuit(next)</code>	วัน 5
จัดสิ่งของลงกล่อง	<code>bin_packing_capa(cap,b,w)</code>	วัน 5
เลือก item maximize profit	0/1 Knapsack หรือ var set	วัน 5
ลด duplicate solutions	Symmetry breaking constraint	วัน 5/6
เพิ่ม pruning speed	Redundant constraints	วัน 6

ข้อผิดพลาดที่พบบ่อย — Effective Modelling

- ✗ `var 0..1000: makespan % hardcoded bound`
- ✓ `int: ub = sum(j,t)(d[j,t]); var lb..ub: makespan;`

Bound ที่ดีคำนวณได้เสมอ — ไม่ต้อง guess

- ✗ `solve minimize cost; % ก่อนทดสอบ satisfy`
- ✓ `solve satisfy; % ทดสอบก่อน, เปลี่ยนเป็น minimize ที่หลัง`

เริ่ม *minimize* โดยไม่ตรวจว่า *model* ถูก → UNSAT หาสาเหตุยาก

- ✗ `% ไม่มี comment อธิบาย constraint`
- ✓ `% Demand: กะเข้าต้องมีพนักงานอย่างน้อย req_morning คน`

Model ที่ไม่มี *comment* อ่านยากมาก โดยเฉพาะเมื่อกลับมาแก้ทีหลัง

- ✗ `forall(i,j in 1..n where i<j)(sym_break[i,j])`
- ✓ `% ตรวจสอบว่า symmetry breaking ไม่ตัด optimal solution ออก`

Symmetry breaking ผิดอาจตัดคำตอบที่ดีออก — ต้องระวัง

Solver ที่รองรับใน MiniZinc

Solver	ประเภท	เหมาะกับ	Flag
Gecode	CP (default)	Integer, Scheduling, Rostering	<code>--solver gecode</code>
Chuffed	CP (lazy)	Scheduling ใหญ่	<code>--solver chuffed</code>
OR-Tools	CP+MIP	Mixed problems	<code>--solver org.minizinc.ortools</code>
HiGHS	MIP (free)	Linear/Integer Programming	<code>--solver highs</code>
COIN-BC	MIP (free)	Linear Programming	<code>--solver cbc</code>
Gurobi	MIP (commercial)	Production LP/MIP ขนาดใหญ่	<code>--solver gurobi</code>

แนะนำสำหรับผู้เริ่มต้น:

Gecode (default) สำหรับปัญหาทั่วไป | Chuffed สำหรับ Scheduling | HiGHS/COIN-BC สำหรับปัญหา Linear

สรุปวันที่ 6 — Effective Modelling

Tight Bounds — solver เร็วขึ้น

lb = min possible | ub = max reasonable | var lb..ub แทน var 0..999

Redundant Constraints — prune เพิ่ม

constraint ที่ซ้ำซ้อนทางตรรกะ แต่ช่วย solver ตัด branch ได้เร็ว

Debugging: 3 ประเภท

UNSAT → เพิ่ม constraint ที่ละอย่าง | Slow → bounds + redundant | Wrong → ตรวจสอบ output มีข้อผิดพลาดหรือไม่

Capstone = นำไปใช้จริง

ปัญหาจริงขององค์กร → MiniZinc model → ผลลัพธ์ที่ดีขึ้น



ยินดีด้วย! ผ่านจบหลักสูตร MiniZinc for Industrial Optimization แล้ว